Sprawozdanie z laboratorium nr 3

Technika cyfrowa (TECY) — projektowanie układów cyfrowych
w programie Logisim Evolution 5

Projekt: lab3.circ (6 układów: zadanie1, bits_check, input_check, grid4x4, display, zadanie5)

Zadania: 5 zadań z zestawu laboratoryjnego TECY (treści zadań: pliki lab3_zadanie1.txt … lab3_zadanie5.txt)

Repozytorium: github.com/01215223/edu_tecy — gałąź arena/7fe48d0e-edu-tecy

Data wykonania: 31.10.2026

Zawartość sprawozdania

Dla każdego z 5 zadań: treść zadania, rozwiązanie analityczne (tabele prawdy, wyrażenia SOP/POS), schemat układu z pliku lab3.circ (wyrenderowany z pliku .circ, kolor przewodów = stan logiczny w symulacji: zielony = 1, niebieski = 0), wyniki weryfikacji symulacyjnej dla wszystkich kombinacji wejść oraz załączone zdjęcia obliczeń odręcznych (pliki PNG).

Spis zadań: 1) układ kombinacyjny zadanie1 (funkcja 4 zmiennych, SOP/POS) · 2) bits_check + input_check (podukłady, funkcja większościowa) · 3) grid4x4 (matryca 16 diod RGBLED) · 4) display (matryca LED 4×5, litery K/U/O/N) · 5) zadanie5 (uproszczenie wyrażenia, f i f2 obok siebie)

1. Wstęp

Celem laboratorium było zaprojektowanie i przetestowanie w programie Logisim Evolution 5 pięciu układów cyfrowych kombinacyjnych, opisanych w plikach lab3_zadanie1.txt … lab3_zadanie5.txt (treść zadań pochodzi z zestawu laboratoryjnego TECY; w repozytorium nie ma osobnego pliku PDF z opisem — treści zadań znajdują się w ww. plikach tekstowych). Wszystkie układy znajdują się w jednym pliku lab3.circ (wersja projektu 5.0.0).

Struktura pliku lab3.circ

Układ (circuit)	Przeznaczenie	Elementy (z pliku .circ)
zadanie1	funkcja M(a,b,c,d) wg równania, bez upraszczania	5 pinów, 4× NOT, 4× AND(3), 1× OR(4)
bits_check	R=1 gdy A ∈ {001,100,111}	2 piny, 1× splitter, 3× NOT, 3× AND(3), 1× OR(3)
input_check	T=1 gdy ≥2 z X,Y,Z ∈ {001,100,111}	4 piny, 3× podukład bits_check, 3× AND(2), 1× OR(3)
grid4x4	matryca 16× RGBLED sterowana przez T (4 bity) i CLR	2 piny, 1× splitter, 8× AND(3), 16× RGBLED
display	matryca LED 4×5: A=00→K, 01→U, 10→O, 11→N	1 pin, 6× splitter, 41× tunnel, 3× NOT, 2× AND, 1× OR, 1× XOR, 1× NOR, 2× Constant, 1× DotMatrix 4×5
zadanie5	f (przed) i f2 (po uproszczeniu) obok siebie	8 pinów, 8× AND, 6× OR, 6× NOT

Metodyka: każdy układ został zweryfikowany symulacyjnie dla wszystkich kombinacji wejść (zadanie1: 16, bits_check: 8, input_check: 512, grid4x4: 16×2, display: 4, zadanie5: 8) — wyniki zestawiono w kolejnych rozdziałach. Dołączono również zdjęcia tabel prawdy i obliczeń wykonanych odręcznie (pliki PNG w repozytorium).

Do renderowania schematów wykorzystano współrzędne elementów i przewodów bezpośrednio z pliku lab3.circ; kolor przewodu pokazuje wartość logiczną w pokazanej symulacji (zielony = 1, niebieski = 0).

2. Zadanie 1 — układ „zadanie1”

Treść (lab3_zadanie1.txt): M(a, b, c, d) = ((b′c) + (a′bd′)) xor ((d((a′+c)′))′). Narysować schemat odpowiadający strukturze równania (bez upraszczania), wyznaczyć tablicę prawdy z kolumnami wyników pośrednich, przetestować wszystkie kombinacje, opisać M w postaci SOP i POS.

Rozwiązanie analityczne

Wyniki pośrednie (zgodnie ze strukturą równania):

```
w1 = b′c     w2 = a′·b·d′     w3 = w1 + w2     w4 = a′ + c
w5 = w4′     w6 = d·w5     w7 = w6′     M = w3 xor w7
```

Tablica prawdy z wynikami pośrednimi (16 wierszy):

a	b	c	d	|	w1	w2	w3	w4	w5	w6	w7	|	M
0	0	0	0	|	0	0	0	1	0	0	1	|	1
0	0	0	1	|	0	0	0	1	0	0	1	|	1
0	0	1	0	|	1	0	1	1	0	0	1	|	0
0	0	1	1	|	1	0	1	1	0	0	1	|	0
0	1	0	0	|	0	1	1	1	0	0	1	|	0
0	1	0	1	|	0	0	0	1	0	0	1	|	1
0	1	1	0	|	0	1	1	1	0	0	1	|	0
0	1	1	1	|	0	0	0	1	0	0	1	|	1
1	0	0	0	|	0	0	0	0	1	0	1	|	1
1	0	0	1	|	0	0	0	0	1	1	0	|	0
1	0	1	0	|	1	0	1	1	0	0	1	|	0
1	0	1	1	|	1	0	1	1	0	0	1	|	0
1	1	0	0	|	0	0	0	0	1	0	1	|	1
1	1	0	1	|	0	0	0	0	1	1	0	|	0
1	1	1	0	|	0	0	0	1	0	0	1	|	1
1	1	1	1	|	0	0	0	1	0	0	1	|	1

Tab. 1. Tablica prawdy M(a,b,c,d) z wynikami pośrednimi w1…w7

Postać SOP (suma mintermów):

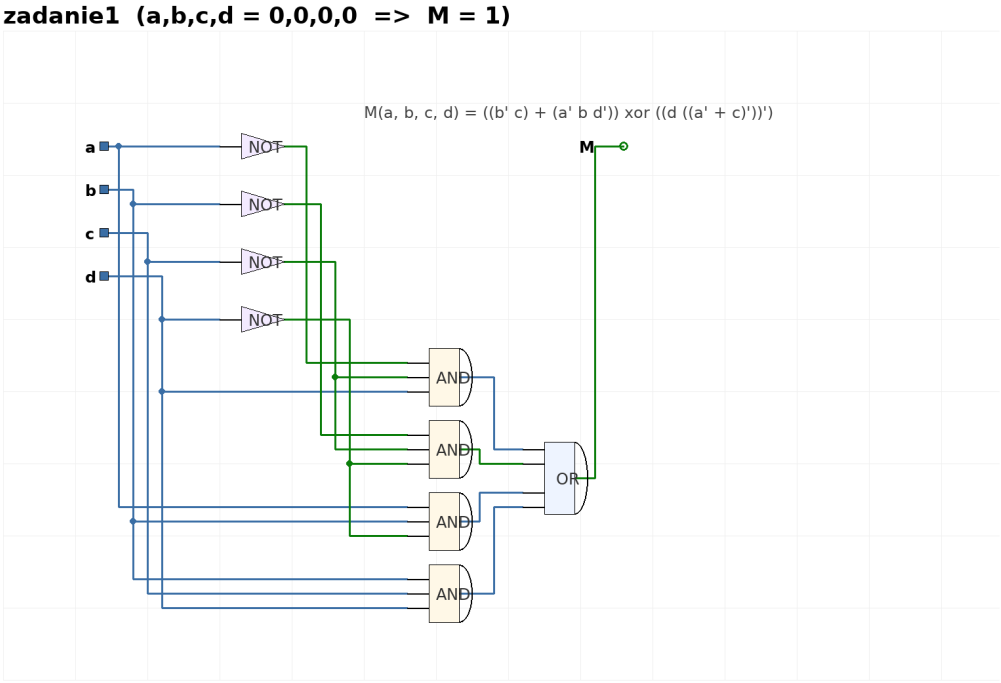
```
M = Σm(0, 1, 5, 7, 8, 12, 14, 15) = a′b′c′d′ + a′b′c′d + a′bc′d + a′bcd + ab′c′d′ + abc′d′ + abcd′ + abcd
```

Postać POS (iloczyn makstermów):

```
M = ΠM(2, 3, 4, 6, 9, 10, 11, 13) = (a+b+c′+d)(a+b+c′+d′)(a+b′+c+d)(a+b′+c′+d)
    (a′+b+c+d′)(a′+b+c′+d)(a′+b+c′+d′)(a′+b′+c+d′)
```

Struktura układu w Logisimie (zadanie1)

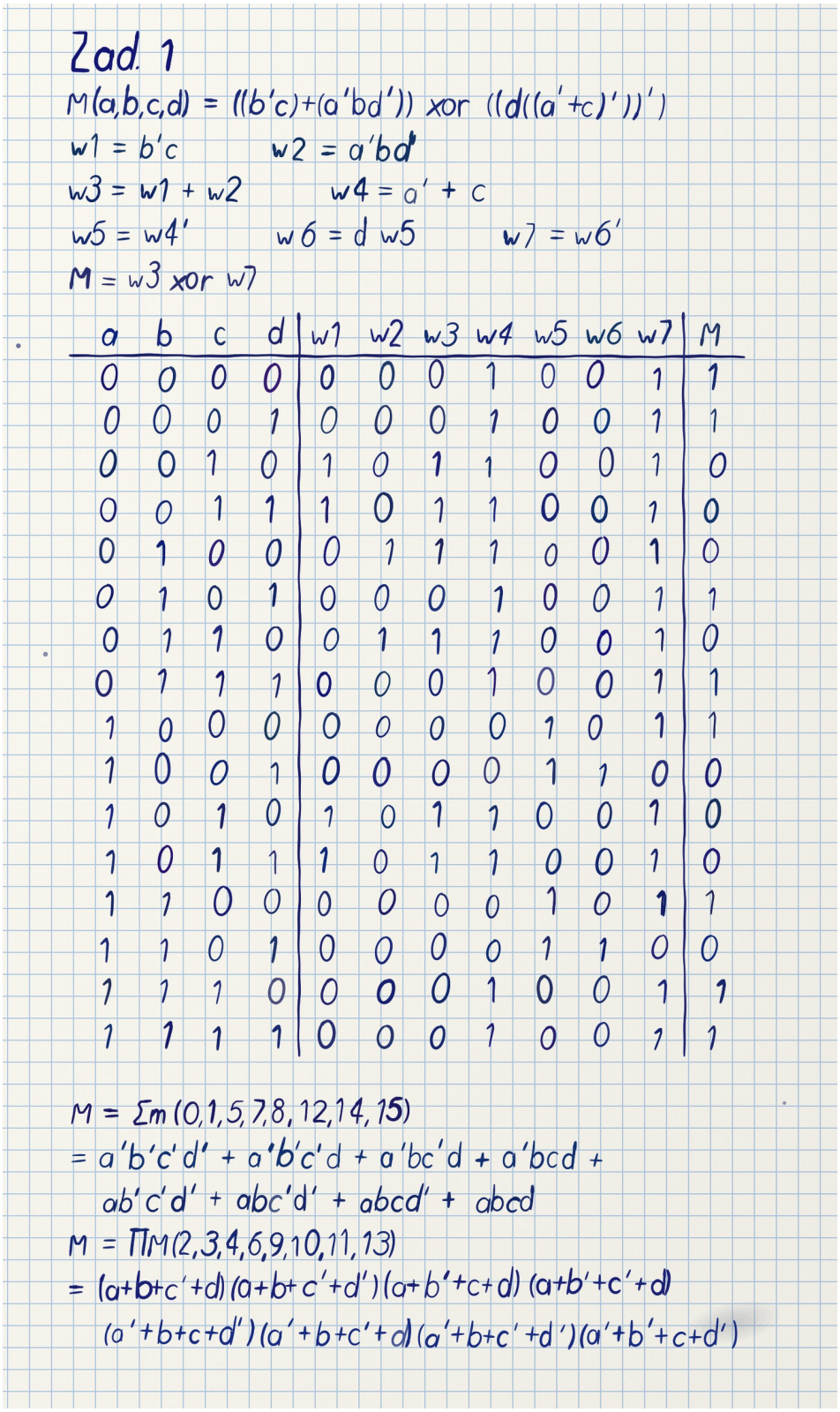
4 piny wejściowe (a, b, c, d), 4 bramki NOT (negacje a′, b′, c′, d′), 4 bramki AND 3-wejściowe (iloczyny pośrednie), 1 bramka OR 4-wejściowa na wyjściu, pin wyjściowy M. Schemat odpowiada strukturze równania — równanie nie było upraszczane.



Rys. 1. Układ zadanie1 (render z lab3.circ) dla a=b=c=d=0 → M=1. Kolor przewodu = stan logiczny (zielony=1, niebieski=0).

Weryfikacja symulacyjna

Układ przetestowano dla wszystkich $2^4 = 16$ kombinacji wejść a,b,c,d. We wszystkich 16 przypadkach wartość M z symulacji była zgodna z tablicą prawdy z kartki (Tab. 1) — zgodność 16/16. Przykładowo: a,b,c,d = 0000 → M = 1; 0010 → M = 0; 1001 → M = 0; 1111 → M = 1.



Zad. 1

$M(a,b,c,d) = ((b'c)+(a'bd')) \text{ xor } ((d((a'+c)'))')$

$w1 = b'c \qquad w2 = a'bd'$

$w3 = w1 + w2 \qquad w4 = a' + c$

$w5 = w4' \qquad w6 = d\,w5 \qquad w7 = w6'$

$M = w3 \text{ xor } w7$

a	b	c	d	w1	w2	w3	w4	w5	w6	w7	M
0	0	0	0	0	0	0	1	0	0	1	1
0	0	0	1	0	0	0	1	0	0	1	1
0	0	1	0	1	0	1	1	0	0	1	0
0	0	1	1	1	0	1	1	0	0	1	0
0	1	0	0	0	1	1	1	0	0	1	0
0	1	0	1	0	0	0	1	0	0	1	1
0	1	1	0	0	1	1	1	0	0	1	0
0	1	1	1	0	0	0	1	0	0	1	1
1	0	0	0	0	0	0	0	1	0	1	1
1	0	0	1	0	0	0	0	1	1	0	0
1	0	1	0	1	0	1	1	0	0	1	0
1	0	1	1	1	0	1	1	0	0	1	0
1	1	0	0	0	0	0	0	1	0	1	1
1	1	0	1	0	0	0	0	1	1	0	0
1	1	1	0	0	0	0	1	0	0	1	1
1	1	1	1	0	0	0	1	0	0	1	1

$M = \Sigma m(0,1,5,7,8,12,14,15)$

$= a'b'c'd' + a'b'c'd + a'bc'd + a'bcd +$
$ab'c'd' + abc'd' + abcd' + abcd$

$M = \Pi M(2,3,4,6,9,10,11,13)$

$= (a+b+c'+d)(a+b+c'+d')(a+b'+c+d)(a+b'+c'+d)$
$(a'+b+c+d')(a'+b+c'+d)(a'+b+c'+d')(a'+b'+c+d')$

Zał. 1. Zdjęcie kartki (plik zadanie1_tabela_odreczna (1).png): równanie, wyniki pośrednie w1…w7, tablica prawdy oraz postacie SOP/POS.

3. Zadanie 2 — układy „bits_check” i „input_check”

Treść (lab3_zadanie2.txt): input_check: trzy 3-bitowe wejścia X, Y, Z i wyjście T; T=1, gdy na co najmniej dwóch wejściach jest jedna z wartości 001, 100, 111. bits_check: wejście 3-bitowe A, wyjście R; R=1 gdy A ∈ {001, 100, 111}. W input_check należy użyć trzech podukładów bits_check. Na kartce: tablice prawdy i wyrażenia boolowskie.

3.1 Podukład bits_check

Wejście A rozbijane jest splitterem na bity A2 (MSB), A1, A0 (LSB). Wyrażenie (postać SOP z tablicy):

`R = Σm(1, 4, 7) = A2′A1′A0 + A2A1′A0′ + A2A1A0`

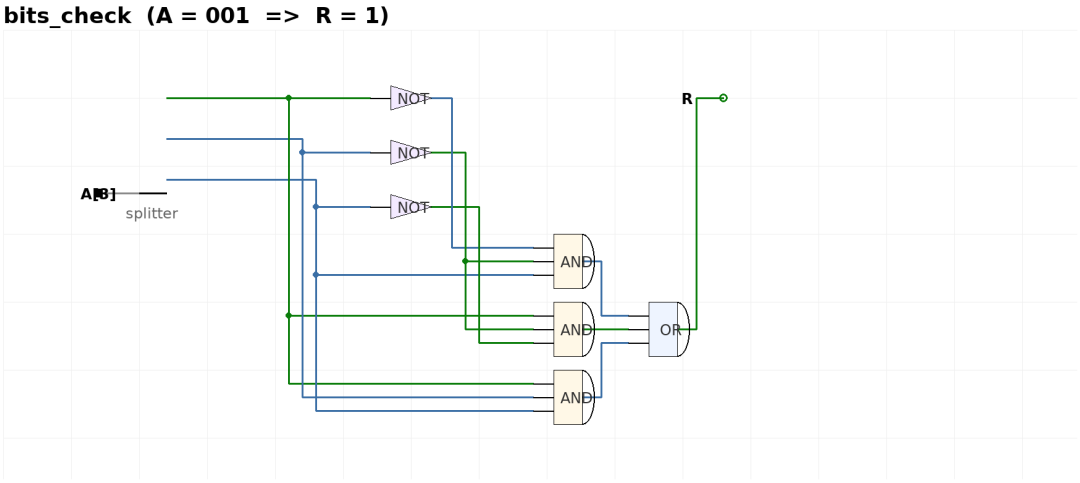
Postać uproszczona (z kartki, mapa Karnaugha): R = A1′·(A2 xor A0) + A2·A1·A0. Postać POS: R = ΠM(0, 2, 3, 5, 6) = (A2+A1+A0)(A2+A1′+A0)(A2+A1′+A0′)(A2′+A1+A0′)(A2′+A1′+A0).

Wyniki pośrednie (wersja z xor, jak na kartce): w1 = A2 xor A0, w2 = A1′, w3 = w2·w1, w4 = A2A1A0, R = w3 + w4.

A2	A1	A0	|	w1	w2	w3	w4	|	R
0	0	0	|	0	1	0	0	|	0
0	0	1	|	1	1	1	0	|	1
0	1	0	|	0	0	0	0	|	0
0	1	1	|	1	0	0	0	|	0
1	0	0	|	1	1	1	0	|	1
1	0	1	|	0	1	0	0	|	0
1	1	0	|	1	0	0	0	|	0
1	1	1	|	0	0	0	1	|	1

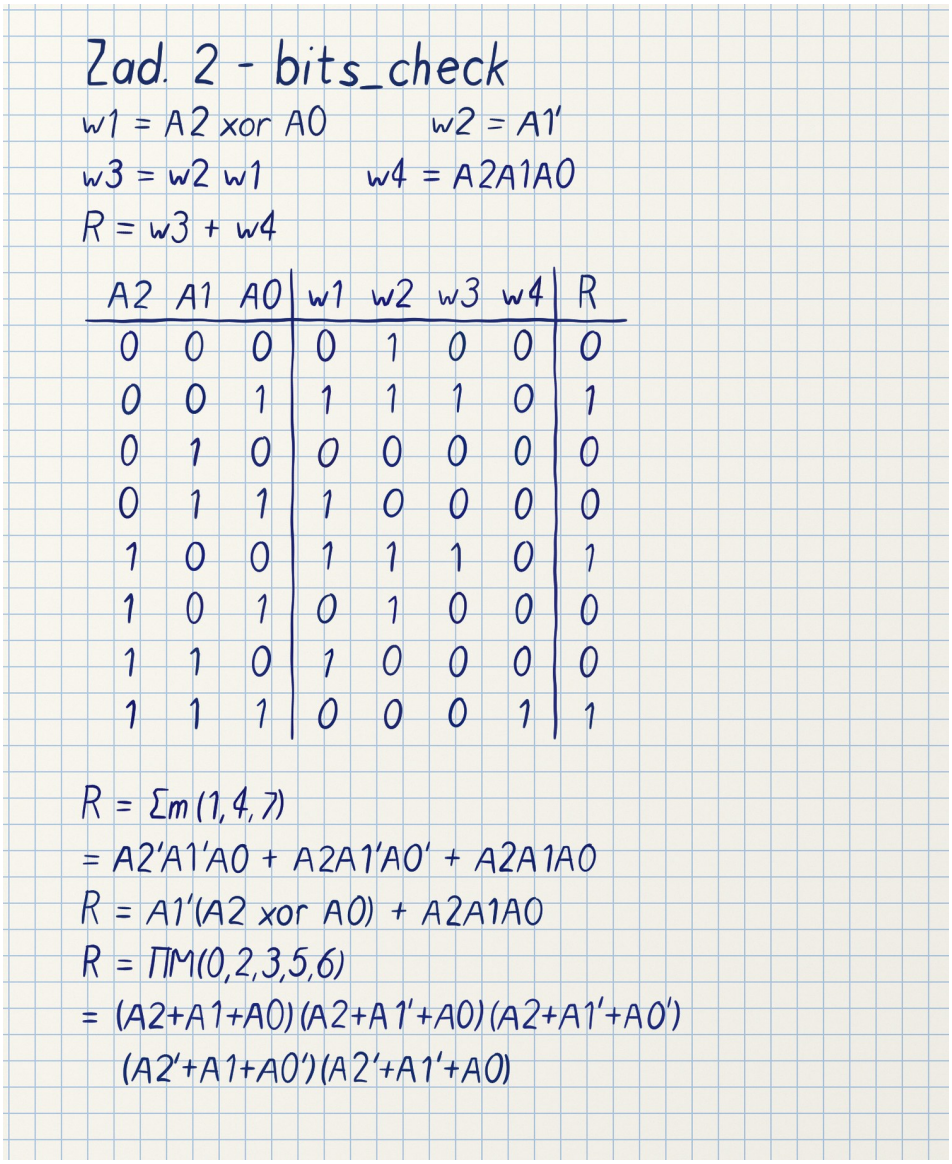
Tab. 2. Tablica prawdy bits_check(A) z wynikami pośrednimi

Struktura w Logisimie: pin A[3] → splitter → 3 bity; 3× NOT, 3× AND 3-wejściowe (mintermy 001, 100, 111), 1× OR 3-wejściowe → pin R. Schemat realizuje wprost postać SOP: R = A2′A1′A0 + A2A1′A0′ + A2A1A0.



Rys. 2. Układ bits_check dla A = 001 → R = 1 (render z lab3.circ).

Weryfikacja: wszystkie $2^3 = 8$ kombinacji A — R=1 dokładnie dla A ∈ {001, 100, 111} — zgodność 8/8.



Zał. 2. Zdjęcie kartki (zadanie2_bits_check_tabela_odreczna.png): wyniki pośrednie, tablica prawdy, SOP/POS dla bits_check.

3.2 Układ input_check

Trzy podukłady bits_check zwracają Rx = bits_check(X), Ry = bits_check(Y), Rz = bits_check(Z). Wyjście T to funkcja większościowa 2 z 3:

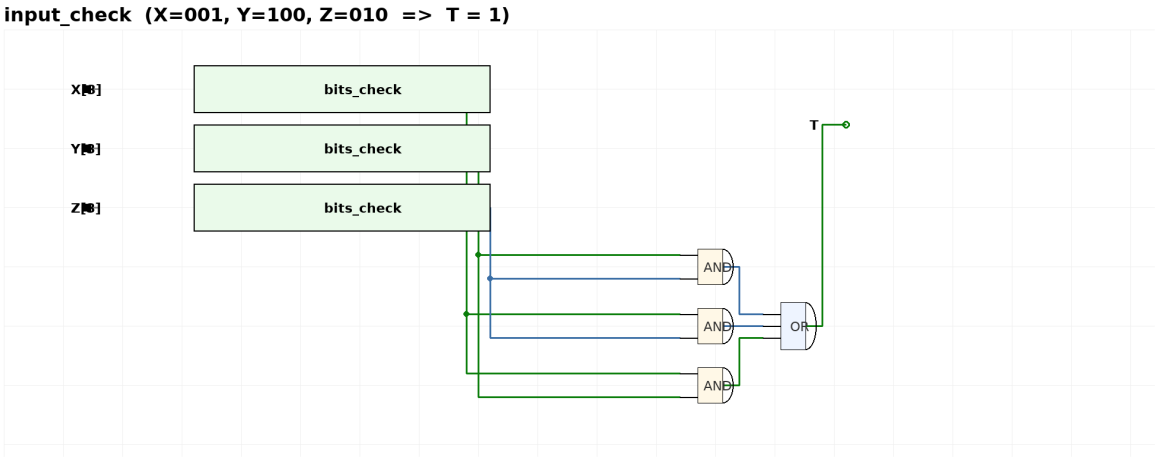
`T = Rx·Ry + Rx·Rz + Ry·Rz (SOP: Σm(3,5,6,7); POS: ΠM(0,1,2,4))`

Wyniki pośrednie: Q1 = RxRy, Q2 = RxRz, Q3 = RyRz, T = Q1 + Q2 + Q3.

Rx	Ry	Rz	|	Q1	Q2	Q3	|	T
0	0	0	|	0	0	0	|	0
0	0	1	|	0	0	0	|	0
0	1	0	|	0	0	0	|	0
0	1	1	|	0	0	1	|	1
1	0	0	|	0	0	0	|	0
1	0	1	|	0	1	0	|	1
1	1	0	|	1	0	0	|	1
1	1	1	|	1	1	1	|	1

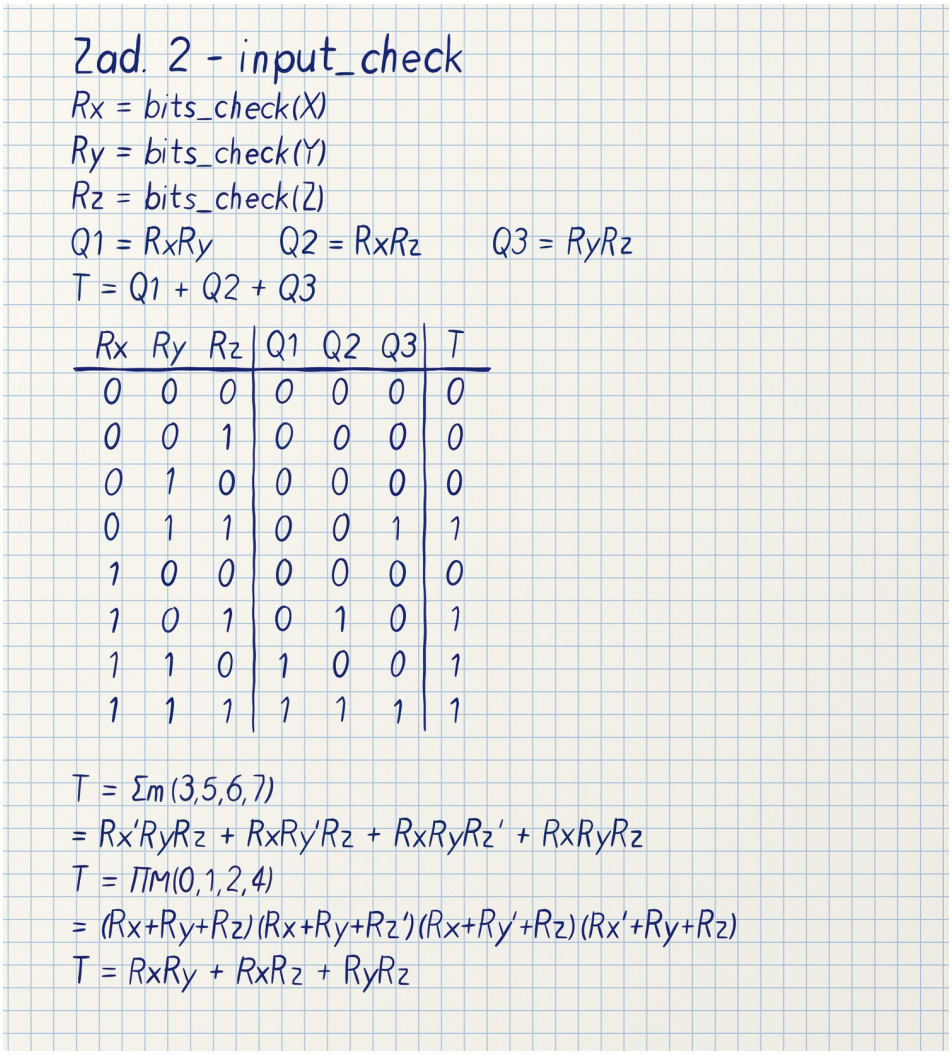
Tab. 3. Tablica prawdy input_check (poziom Rx, Ry, Rz) z wynikami pośrednimi

Struktura w Logisimie: piny X[3], Y[3], Z[3] → trzy instancje podukładu bits_check → 3× AND 2-wejściowe (iloczyny par RxRy, RxRz, RyRz) → 1× OR 3-wejściowe → pin T.



Rys. 3. Układ input_check dla X=001, Y=100, Z=010 → Rx=Ry=1, Rz=0 → T = 1 (render z lab3.circ).

Weryfikacja: wszystkie $2^9 = 512$ kombinacji (X, Y, Z). T=1 dokładnie wtedy, gdy co najmniej dwa z Rx, Ry, Rz są równe 1, czyli gdy co najmniej dwa wejścia należą do {001, 100, 111} — zgodność 512/512. Przykłady: X=001, Y=100, Z=000 → T=1; X=010, Y=101, Z=110 → T=0.



Zał. 3. Zdjęcie kartki (zadanie2_input_check_tabela_odreczna.png): tablica prawdy i wyrażenia dla input_check.

4. Zadanie 3 — układ „grid4x4” (matryca 16 diod RGBLED)

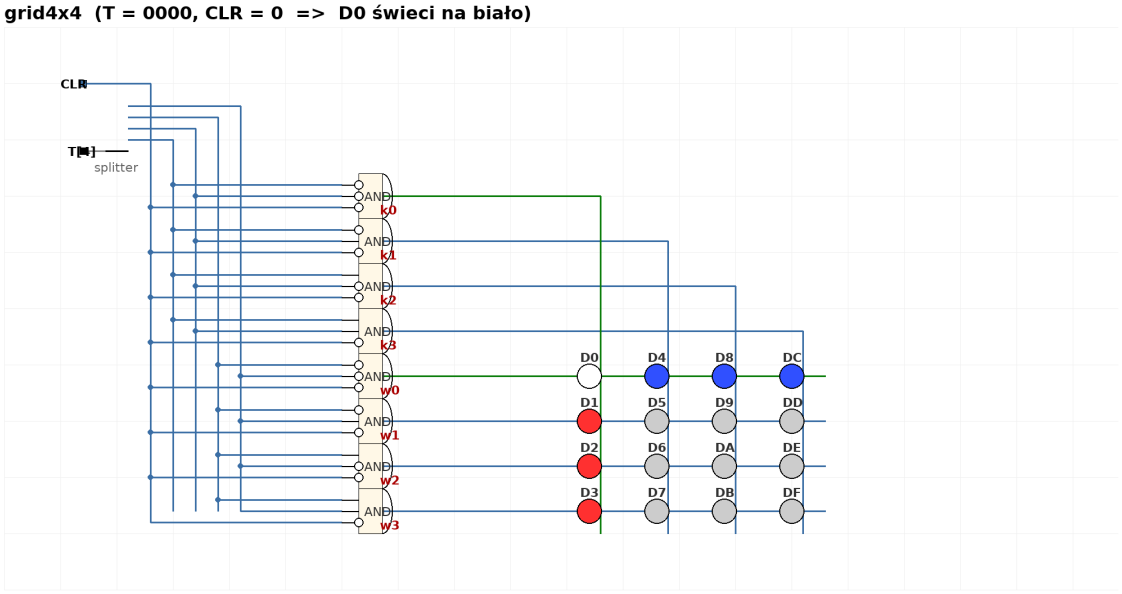
Treść (lab3_zadanie3.txt): układ grid4x4 z 4-bitowym wejściem T, wejściem CLR i 16 diodami RGBLED w siatce 4×4 (przewody poziome/pionowe przechodzą przez diody i nie są ze sobą połączone). Dioda świeci na czerwono przy aktywnej linii poziomej, na niebiesko przy pionowej, na biało przy obu. CLR=1 → wszystkie diody zgaszone. Ograniczenie: nie więcej niż 8 bramek, jak najmniej negatorów.

Rozwiązanie

Wejście T (4 bity: T3…T0, T3 = MSB) rozbijane jest splitterem. Starsze dwa bity wybierają kolumnę, młodsze dwa — wiersz. Dokładnie 8 bramek AND 3-wejściowych z zanegowanymi wejściami (bąbelki negacji na wejściach bramek — dzięki temu w układzie nie ma ani jednego osobnego negatora):

```
kolumny:  k0 = T3′·T2′·CLR′   k1 = T3′·T2·CLR′   k2 = T3·T2′·CLR′   k3 = T3·T2·CLR′
wiersze:  w0 = T1′·T0′·CLR′   w1 = T1′·T0·CLR′   w2 = T1·T0′·CLR′   w3 = T1·T0·CLR′
```

Linia kolumny k_c biegnie pionowo przez diody kolumny c (czerwony), linia wiersza w_r poziomo przez diody wiersza r (niebieski); na skrzyżowaniu stoi dioda RGBLED — świeci biało, gdy obie linie aktywne. Dioda D(4c+r) na pozycji (kolumna c, wiersz r).



Rys. 4. Układ grid4x4 dla T = 0000, CLR = 0: k0 = w0 = 1 → dioda D0 świeci na biało, D1–D3 na czerwono (kolumna 0), D4/D8/DC na niebiesko (wiersz 0), pozostałe zgaszone (render z lab3.circ).

Weryfikacja symulacyjna (wszystkie 16 wartości T, CLR=0 oraz CLR=1):

T (T3T2T1T0)	CLR	aktywna kolumna	aktywny wiersz	świecąca dioda (kolor)
0000	0	k0	w0	D0 (biały)
0101	0	k2	w2	DA (biały)
1010	0	k1	w1	D5 (biały)
1111	0	k3	w3	DF (biały)
0000	1	—	—	wszystkie zgaszone (CLR=1)

Tab. 4. Wybrane stany grid4x4 z symulacji (pełna zgodność dla wszystkich 16 wartości T; CLR=1 zeruje wszystkie k i w)

Dla T = 0000 (przykład z treści zadania) zapala się dokładnie dioda 0 — na biało, co zgadza się z rysunkiem w treści zadania. Ograniczenia spełnione: dokładnie 8 bramek, 0 negatorów.

5. Zadanie 4 — układ „display” (matryca LED 4×5: K/U/O/N)

Treść (lab3_zadanie4.txt): układ display z 2-bitowym wejściem A i matrycą LED Matrix 4 kolumny × 5 wierszy (Input Format = Rows). A=00 → „K”, 01 → „U”, 10 → „O”, 11 → „N”. Matrycę steruje się przez 5 magistral wierszowych (po 4 bity), rozbitych splitterami na 20 pojedynczych przewodów.

Rozwiązanie — dekoder adresu A

Wejście A[2] (bity A1, A0) rozbite splitterem; mały dekoder (3× NOT, 2× AND, 1× OR, 1× XOR, 1× NOR + stałe ONE=1, ZERO=0) generuje sygnały pomocnicze (tunele):

```
P = A1·A0′ (A=10)   nP = P′   Q = A1·A0 (A=11)   R = (A1+A0)′ = A1′A0′ (A=00)
S = A1+A0   XO = A1⊕A0   XN = (A1⊕A0)′   ONE = 1   ZERO = 0
```

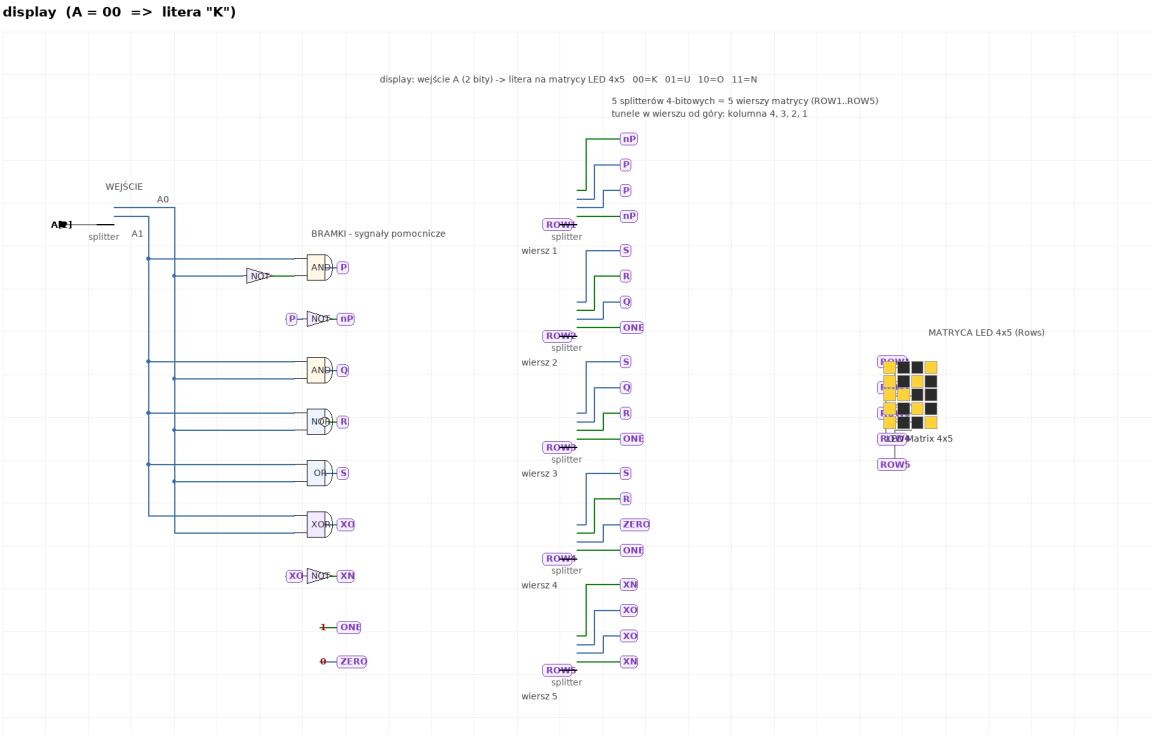
Każdy wiersz matrycy (magistrala 4-bitowa ROW1…ROW5) składa się z 4 pikseli podanych przez tunele (bit 3 = piksel lewy … bit 0 = piksel prawy):

```
ROW1 = nP P P nP   ROW2 = S R Q ONE   ROW3 = S Q R ONE   ROW4 = S R S ONE   ROW5 = XN XO XO XN
```

Wzorce 4×5 (X = LED włączony, · = wyłączony) uzyskane z symulacji układu:

A (A1A0)	litera	wzorzec 4×5 (z symulacji lab3.circ)
A=00	K	X..X .X.X XX.. .X.X X..X
A=01	U	.XX. X..X X..X X..X .XX.
A=10	O	X..X X..X X..X X..X .XX.
A=11	N	X..X XX.X X.XX X..X X..X

Tab. 5. Wzorce wyświetlane na matrycy LED 4×5 dla kolejnych wartości A (weryfikacja symulacyjna 4/4)



Rys. 5. Układ display dla A = 00 — na matrycy LED 4×5 wyświetlana jest litera „K” (render z lab3.circ; żółte piksele = LED włączone).

6. Zadanie 5 — układ „zadanie5” (uproszczenie wyrażenia f(a,b,c))

Treść (lab3_zadanie5.txt): f(a, b, c) = abc′ + abc + ((a+b)(bc′)) + (((a′+b′)′)c). Uprościć wyrażenie na kartce do postaci sumy iloczynów, a w Logisim-evolution narysować w jednym układzie zadanie5 dwa schematy: f (struktura przed uproszczeniem) i f2 (po uproszczeniu, jak najmniej bramek), oraz sprawdzić wyniki w symulacji.

Uproszczenie (krok po kroku, algebra Boole’a)

```
(a′+b′)′ = a·b   (De Morgan: (x+y)′ = x′y′, tu x=a′, y=b′ ⇒ (a′+b′)′ = a·b)
((a+b)(bc′)) = a·b·c′ + b·b·c′ = abc′ + bc′
f = abc′ + abc + abc′ + bc′ + abc = abc′ + abc + bc′ = bc′ + ab·(c′+c) = bc′ + ab
f2(a,b,c) = ab + bc′ = b·(a + c′)
```

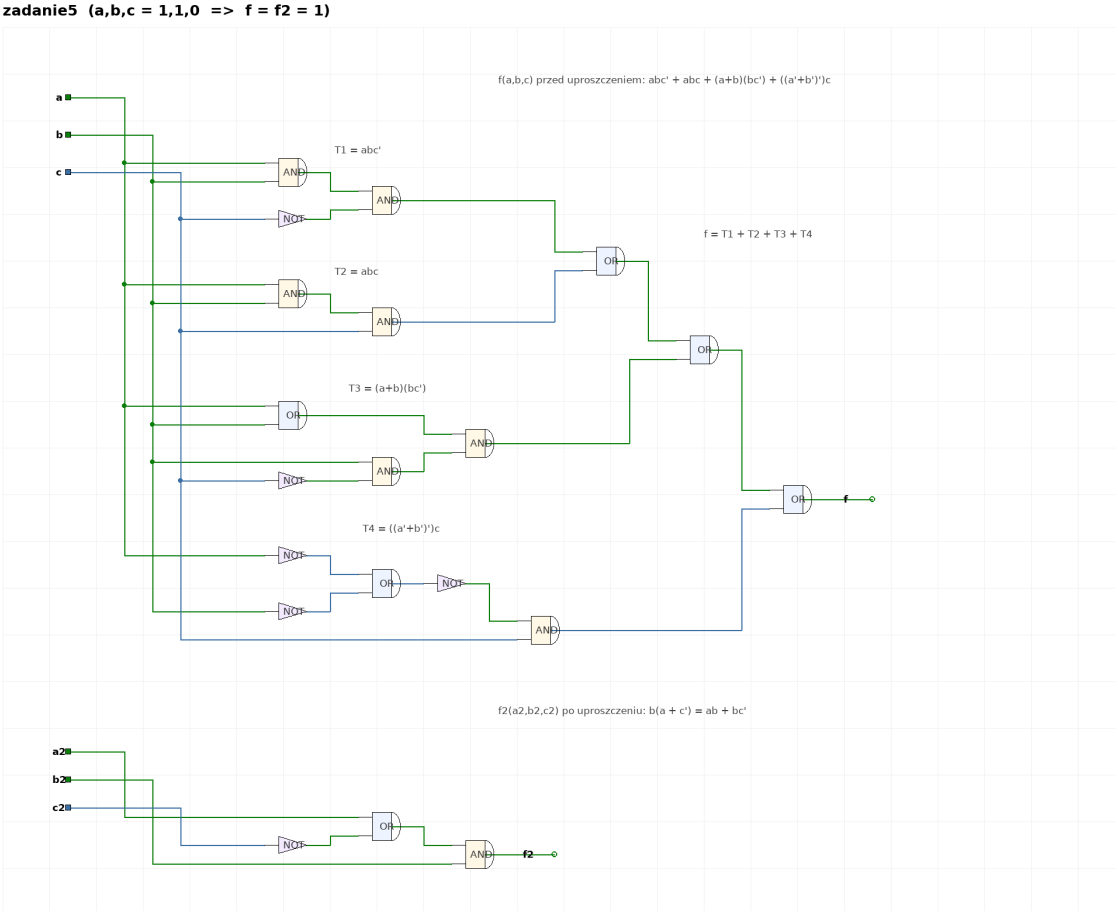
Tablica prawdy (T1…T4 — składniki równania; f — przed uproszczeniem; f2 — po uproszczeniu):

a	b	c	|	T1=abc′	T2=abc	T3=(a+b)bc′	T4=((a′+b′)′)c	|	f	f2=ab+bc′
0	0	0	|	0	0	0	0	|	0	0
0	0	1	|	0	0	0	0	|	0	0
0	1	0	|	0	0	1	0	|	1	1
0	1	1	|	0	0	0	0	|	0	0
1	0	0	|	0	0	0	0	|	0	0
1	0	1	|	0	0	0	0	|	0	0
1	1	0	|	1	0	1	0	|	1	1
1	1	1	|	0	1	0	1	|	1	1

Tab. 6. Tablica prawdy f i f2 — f = f2 dla wszystkich 8 kombinacji (8/8)

Struktura układu w Logisimie

W jednym układzie zadanie5 znajdują się dwa tory: górny realizuje f w strukturze podanego równania (oznaczenia na schemacie: T1=abc′, T2=abc, T3=(a+b)(bc′), T4=((a′+b′)′)c, f = T1+T2+T3+T4 — 8× AND, 6× OR, 6× NOT), dolny realizuje f2 = ab + bc′ możliwie najmniejszą liczbą bramek (2× AND + 1× OR + 1× NOT). Wejścia a,b,c oraz a2,b2,c2, wyjścia f i f2.



Rys. 6. Układ zadanie5 dla a,b,c = 1,1,0 → f = f2 = 1 (render z lab3.circ; góra: tor f, dół: tor f2).

Weryfikacja symulacyjna: dla wszystkich $2^3 = 8$ kombinacji zachodzi f = f2 = wartość z równania wyjściowego — zgodność 8/8.

7. Podsumowanie i wnioski

Wszystkie pięć zadań zrealizowano w jednym pliku lab3.circ (Logisim Evolution 5) i zweryfikowano symulacyjnie dla wszystkich kombinacji wejść:

Zadanie	Układ(y)	Zakres testu	Wynik
1	zadanie1	16 kombinacji a,b,c,d	M zgodne z tablicą z kartki 16/16; SOP=Σm(0,1,5,7,8,12,14,15), POS=ΠM(2,3,4,6,9,10,11,13)
2	bits_check + input_check	8 kombinacji A; 512 kombinacji X,Y,Z	R=1 tylko dla {001,100,111} 8/8; T=większość 2z3 512/512
3	grid4x4	16 wartości T × CLR∈{0,1}	T=0000 → D0 biała; CLR=1 → wszystko zgaszone; 8 bramek AND, 0 negatorów
4	display	4 wartości A	A=00→K, 01→U, 10→O, 11→N (4/4)
5	zadanie5 (f i f2)	8 kombinacji a,b,c	f = f2 = ab + bc′ 8/8

Tab. 7. Zestawienie wyników weryfikacji

Wnioski: (1) podukłady w Logisim-evolution (bits_check wewnątrz input_check) pozwalają budować układy hierarchicznie; (2) negacje na wejściach bramek (bąbelki) pozwoliły zrealizować grid4x4 dokładnie 8 bramkami bez osobnych negatorów; (3) funkcję większościową 2 z 3 (T = RxRy + RxRz + RyRz) otrzymano bezpośrednio z tablicy prawdy poziomu Rx,Ry,Rz; (4) uproszczenie f do postaci f2 = ab + bc′ zmniejszyło liczbę bramek z 20 do 4 przy identycznej tablicy prawdy; (5) matrycę LED 4×5 można sterować 5 magistralami wierszowymi rozbitymi splitterami na 20 bitów.

8. Załączniki (pliki w repozytorium)

• lab3.circ — projekt Logisim Evolution 5 (6 układów: zadanie1, bits_check, input_check, grid4x4, display, zadanie5)
• lab3_zadanie1.txt … lab3_zadanie5.txt — treści 5 zadań
• zadanie1_tabele.txt, zadanie2_bitscheck_tabela.txt, zadanie2_inputcheck_tabela.txt — tabele prawdy i wyrażenia SOP/POS
• zadanie1_tabela_odreczna (1).png — kartka z zadaniem 1 (zał. 1 w sprawozdaniu)
• zadanie2_bits_check_tabela_odreczna.png — kartka bits_check (zał. 2)
• zadanie2_input_check_tabela_odreczna.png — kartka input_check (zał. 3)
• Rys. 1–6 — schematy wyrenderowane z pliku lab3.circ (stany symulacji oznaczone kolorami przewodów)